System Architecture & Code Lifecycle Specification

This document details the core technical topology, database schema, and redemption token lifecycle for the Bantaba Cash-First Remittance Operating System. Designed for high availability and offline resiliency across emerging market physical agent networks.

1. High-Level Architecture Topology

> **Core Components:** Edge POS Client ➔ Gateway REST API (TLS 1.3) ➔ Remittance Engine Microservice ➔ Isolation Token Vault ➔ PostgreSQL Double-Entry General Ledger.

- **Edge POS Client:** Lightweight HTML/JS application running on low-power counter terminals or mobile POS tablets with local SQLite sync queue.
- **Remittance Engine API:** Stateless Node.js / Go microservice handling currency conversion calculations, fee lookups, and transaction orchestration.
- **Token Vault Service:** Cryptographically isolated service responsible for generating, hashing, and validating 10-digit redemption tokens.

2. 10-Digit Redemption Code Lifecycle

The redemption token is a formatted 10-character alphanumeric code (e.g., `BNT-9042-18`) formatted as `BNT-XXXX-XX` using an ambiguous-character-free Base32 alphabet (omitting 0, 1, I, O to prevent cashier entry error).

State	Trigger Action	Ledger & Vault Effect
1. GENERATED	Cashier collects cash at origin desk and submits transaction.	Token hashed (HMAC-SHA256) & stored. Double-entry booked: `DR 1010 Vault_Cash / CR 2010 Cust_Payable`.
2. ACTIVE_UNCLAIMED	Token transmitted to recipient via SMS/WhatsApp or paper slip.	Token searchable via hashed lookup. Valid for 14 calendar days.
3. VERIFIED	Payout cashier enters code + recipient phone number.	System verifies phone match & lock token for 5 minutes during physical ID check.
4. REDEEMED	Payout cashier confirms ID & disburses cash.	Token marked `EXPIRED_REDEEMED`. Ledger booked: `DR 2010 Cust_Payable / CR 1010 Vault_Cash`.
5. VOID_EXPIRED	Token unredeemed after 14 days or canceled by sender.	Reversal entry booked back to origin sender balance account.

3. Database Core Schema (PostgreSQL DDL)

```
CREATE TABLE transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    reference_no VARCHAR(16) UNIQUE NOT NULL,
    redemption_token_hash VARCHAR(64) NOT NULL,
    origin_branch_id VARCHAR(12) NOT NULL,
    dest_branch_id VARCHAR(12) NOT NULL,
    sender_name VARCHAR(100) NOT NULL,
    sender_phone VARCHAR(20) NOT NULL,
    recipient_name VARCHAR(100) NOT NULL,
    recipient_phone VARCHAR(20) NOT NULL,
    send_currency VARCHAR(3) NOT NULL,
    send_amount NUMERIC(12, 2) NOT NULL,
    counter_fee NUMERIC(8, 2) NOT NULL,
    fx_rate NUMERIC(10, 4) NOT NULL,
    payout_currency VARCHAR(3) NOT NULL,
    payout_amount NUMERIC(14, 2) NOT NULL,
    status VARCHAR(20) DEFAULT 'UNREDEEMED' CHECK (status IN ('UNREDEEMED', 'REDEEMED',
'EXPIRED', 'CANCELLED')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    redeemed_at TIMESTAMP WITH TIME ZONE,
    redeemed_by_staff_id VARCHAR(20)
);

CREATE TABLE journal_entries (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    transaction_id UUID REFERENCES transactions(id),
    account_code VARCHAR(10) NOT NULL,
    debit_amount NUMERIC(14, 2) DEFAULT 0.00,
    credit_amount NUMERIC(14, 2) DEFAULT 0.00,
    currency VARCHAR(3) NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

4. Security, Fraud Control & Rate Limiting

- **HMAC-SHA256 Token Storage:** Plaintext 10-digit codes are never stored in the database. Only HMAC hashes are saved.
- **Brute-Force Lockout:** Maximum 3 failed lookup attempts per recipient phone number per hour before locking account lookup.
- **Two-Factor Phone Matching:** Redemption code lookup requires exact match with registered recipient phone number.